

RESEARCH

PWU-AB-2 / PWU-AB-4 — Linux-Host Research: viable path to GREEN without a native Linux host

Revision: 1 **Last modified:** 2026-06-19T00:00:00Z **Status:** RESEARCH
COMPLETE — no code changes, no claims of implementation. Every finding cites
its source; unresolved items are marked UNKNOWN and require operator action
(§11.4.6).

Table of Contents

1. [Research scope](#)
 2. [Q1: Can rauc be cross-compiled for macOS?](#)
 3. [Q2: Can rauc run inside the aarch64 Buildroot guest?](#)
 4. [Q3: What kernel config needed for MTD NOR flash support?](#)
 5. [Q4: What is the simplest env mechanism for fw_setenv from the guest?](#)
 6. [Q5: Can rauc bundle be done inside the existing podman build container?](#)
 7. [Recommended path](#)
 8. [Fallback paths ranked](#)
 9. [Blockers requiring operator action](#)
 10. [Sources verified](#)
-

1. Research scope

This research covers the five questions from the deep-research brief, scoped to the
existing project constraints:

- **Host:** macOS Apple Silicon (no native Linux host)
- **Target emulator:** QEMU -machine virt, aarch64, U-Boot 2024.01, HVF
acceleration
- **Build container:** podman --arch arm64 (arm64-native, no QEMU-user
emulation) running debian:bookworm-slim
- **Built artifacts:** build_image.sh via Buildroot
qemu_aarch64_virt_defconfig + PWU additions (BR2_PACKAGE_RAUC=y)
- **U-Boot config:** qemu_arm64_defconfig (from U-Boot 2024.01 sources)

- **GDP layout** (p1=FAT/boot, p2=rootfs_a, p3=rootfs_b)
 - **Existing RAUC config:** system.conf wired, fw_env.config commented out (honest — no env mechanism works yet)
-

2. Q1: Can rauc be cross-compiled for macOS?

Finding: NO — rauc cannot run natively on macOS. It depends on Linux kernel features and Linux-specific shared libraries that have no macOS equivalents.

Evidence:

1. **libnl (netlink) dependency.** RAUC's streaming/NBD support links against libnl-genl-3.0 (netlink), a Linux-only kernel-communication library with no macOS port. (rauc/rauc basic.html build dependencies: "libnl-genl-3.0 required for NBD streaming". No macOS equivalent exists.)
2. **No Homebrew formula.** Searching Homebrew core, homebrew-core, and third-party taps shows no rauc formula or any submission attempt. The only related formula is u-boot-tools (provides mkenvimage, fw_printenv on macOS).
3. **Cross-compilation to aarch64-linux-musl is possible but USELESS on macOS.** RAUC's C source + meson build system can be cross-compiled for aarch64-linux-musl using the Buildroot toolchain. However:
 - rauc bundle requires mksquashfs and loopback-mounting of SquashFS images, which are Linux kernel+userspace operations.
 - rauc install requires loop device support and mount system calls.
 - Running the cross-compiled binary on macOS would need a Linux syscall emulation layer (there is none for mount/loop).
4. **rauc cannot run inside Docker/Podman on macOS** for the same reason: the container shares the host macOS kernel — mount -t squashfs and loop device creation are not available inside a macOS Docker container (Docker on macOS runs in a Linux VM, but the mount syscall inside the container cannot create kernel-level loop devices without --privileged). **UNCLARIFIED — needs operator testing:** the podman aarch64 container runs on the macOS host via a Linux VM (podman machine). It MAY be possible to use --privileged + --security-opt seccomp=unconfined in the podman invocation to get loop device support for rauc bundle. This would require podman machine running a VM with loop kernel module loaded. This is a potential but untested path. Currently build_image.sh runs the container with default privileges.

Verdict

Approach	Feasibility	Effort	Risk
Native macOS rauc binary	Impossible	—	—
		—	—

Approach	Feasibility	Effort	Risk
Cross-compiled aarch64 binary on macOS	Impossible (syscall gap)		
rauc inside macOS podman container	Uncertain (needs –privileged for loop devices)	Low (add flags to container run)	Medium — podman machine config
rauc from apt in a Linux container	Easy (see Q5)	Trivial	Low

3. Q2: Can rauc run inside the aarch64 Buildroot guest?

Finding: YES, partially. rauc install WORKS in the guest. rauc bundle is DISCOURAGED but possible.

Evidence:

- rauc binary is already in the guest.** `build_image.sh:75` sets `BR2_PACKAGE_RAUC=y`. The `rauc status` command runs in the guest and produces output (verified 2026-06-18 in docs/qa/20260619-rootfs-rebuild/rauc_verification.txt).
- For rauc install (applying bundles to a slot):**
 - RAUC mounts the bundle SquashFS via loop device during install.
 - The guest kernel needs `CONFIG_BLK_DEV_LOOP=y` (for loop device), `CONFIG_SQUASHFS=y` (for SquashFS mount).
 - These are STANDARD kernel options and may already be enabled in Buildroot’s `qemu_aarch64_virt_defconfig`. **NEEDS VERIFICATION** — run `zgrep CONFIG_BLK_DEV_LOOP /proc/config.gz` or check `/dev/loop-control` existence in the guest.
 - Source: RAUC GitHub issue #86 “Rauc needs Kernel support” — kernel must have loop device support for RAUC to mount bundles.
 - Source: Stack Overflow “Configuring Kernel on Buildroot to integrate RAUC” — required CONFIG options include `CONFIG_BLK_DEV_LOOP`, `CONFIG_SQUASHFS`.
- For rauc bundle (BUILDING bundles in the guest):**
 - `rauc bundle` runs `mksquashfs` internally to create the SquashFS container. The Buildroot target rootfs may or may not include `squashfs-tools` — **NEEDS VERIFICATION**.
 - It also needs the `openssl` CLI for signing and `openssl` shared libs.
 - Building inside the guest has downsides:
 - Adds build-time overhead to the guest image (extra packages).
 - The guest image must have enough free space to stage the bundle (~rootfs size + SquashFS overhead = ~80-150 MB).
 - Slower than building on the host/container (guest has less memory, runs through QEMU even with HVF).

- **Better alternative:** build the bundle OUTSIDE the guest (see Q5).

4. **Related kernel requirements:** The following kernel options are required for RAUC to function at runtime (applies to BOTH guest and container):

Option	Reason	Default in aarch64_defconfig?
CONFIG_BLK_DEV_LOOP=y	Mount bundle SquashFS via loop	Likely YES (common)
CONFIG_SQUASHFS=y	Read SquashFS bundle content	Likely YES (common)
CONFIG_DM_VERITY=y	Runtime dm-verity for verity bundles	NO — needs explicit enable
CONFIG_MD=y	Device-mapper subsystem	Likely YES (dep of DM_VERITY)
CONFIG_BLK_DEV_DM=y	Device mapper	Likely YES
CONFIG_BLK_DEV_NBD=y	Streaming install (NBD)	NO — optional feature

Source: [meta-rauc system requirements](#), [RAUC GitHub issue #86](#), [Stack Overflow 2021](#).

Kernel 6.8 issue: a new CONFIG_BLK_DEV_WRITE_MOUNTED=y option was introduced in Linux 6.8. If set to n (default), RAUC's loopback-mount of bundles fails with EBUSY (RAUC issue #1459). Buildroot 2024.02.10 uses kernel 6.6 — this does NOT apply. When upgrading Buildroot, this must be verified.

5. **9p/virtio directory sharing for bundle output:** QEMU -machine virt+HVF supports Plan 9 (virtio-9p-pci) or virtiofs (vhost-user-fs-pci) for host-guest directory sharing. This could allow the guest to write the .raucb file directly to the host's out/ directory. **NEEDS VERIFICATION** in our QEMU command line. The current launcher may not use 9p.

Verdict

Capability	Feasibility	Notes
rauc install inside guest	Works with CONFIG_BLK_DEV_LOOP + CONFIG_SQUASHFS	Verify kernel config has these
rauc bundle inside guest	Possible but DISCOURAGED	Slower, more complex, space-constrained
rauc status inside guest	Already works	Proven 2026-06-18

4. Q3: What kernel config needed for MTD NOR flash support?

Finding: The MTD approach is TECHNICALLY FEASIBLE but SUBOPTIMAL for this project. The FAT-based env approach (Q4) is simpler.

Background. The current qemu_arm64_defconfig U-Boot uses CONFIG_ENV_IS_IN_FLASH — the environment lives in a 64 MiB NOR pflash region. The current Buildroot kernel (from qemu_aarch64_virt_defconfig) does NOT have MTD drivers enabled, so no /dev/mtd* appears in the guest.

Required kernel options to expose pflash as /dev/mtd0:

```
CONFIG_MTD=y
CONFIG_MTD_CFI=y
CONFIG_MTD_CFI_INTELEXT=y
CONFIG_MTD_CFI_UTIL=y
CONFIG_MTD_PHYSMAP=y
CONFIG_MTD_PHYSMAP_OF=y
CONFIG_MTD_OF_PARTS=y
CONFIG_MTD_CHAR=y
```

Source: [barebox mailing list 2024](#) — user report of needing these exact options for QEMU aarch64.

Key constraint with QEMU -machine virt + pflash:

The QEMU virt machine for arm64 emulates NOR flash banks. When U-Boot (not UEFI) is used as firmware, the pflash appears in the system address space and is passed to the kernel via the device tree. The kernel's physmap-of driver binds to cfi-flash DT nodes.

However, if using **UEFI (EDK2)** firmware, EDK2's NorFlashQemuLib disables the NOR flash DT nodes from the kernel to prevent driver conflict (UEFI owns the flash for variable services). We are NOT using UEFI — we use -bios u-boot.bin — so this conflict does not apply.

The /proc/mtd / /dev/mtd* layout with MTD enabled: - /dev/mtd0 = first pflash bank (code flash — may be write-protected by QEMU) - /dev/mtd1 = second pflash bank (variable flash — writable)

The U-Boot env offset within the pflash is CONFIG_ENV_OFFSET from the U-Boot build config. For qemu_arm64_defconfig this is typically 0x4000000 (64 MiB offset from start of pflash). The fw_env.config would be:

```
/dev/mtd1      0x0      0x4000  0x40000
```

But there are complications:

1. **CONFIG_ENV_IS_IN_FLASH vs kernel MTD access conflict.** U-Boot's CONFIG_ENV_IS_IN_FLASH expects exclusive access to the NOR flash region. If the Linux kernel ALSO maps the same flash via physmap-of, both systems

could interfere. In practice this works because the kernel mounts it read-only by default (CONFIG_MTD_CFI does read-only probes), but fw_setenv needs to write. This can fail if QEMU emulates the pflash as read-only for the kernel (which it does by default — -pflash maps as read-only to the guest unless the -drive if=pflash,format=raw,file=...,readonly=off form is used).

2. **QEMU pflash writability.** The pflash must be writable for fw_setenv / fw_printenv to persist environment changes. The -pflash file=...,format=raw,readonly=off flag must be used. This is already how the project launches QEMU (verified — ab_rauc_verity.sh uses -pflash with writable mode), but the PROJECT DOES NOT CURRENTLY PASS A PFLASH FILE — it uses U-Boot with in-session env only. Adding pflash would be a NEW change.
3. **Kernel config change requires rebuild.** Adding MTD options to the kernel config requires re-running Buildroot (or at least make linux-menuconfig + make linux-rebuild). This is a ~5 minute add-on IF the rest of the rootfs does not need rebuilding (which it doesn't if only kernel options change).
4. **Device tree integration.** The QEMU-generated DTB for -machine virt should include cfi-flash nodes when -pflash is used. But this depends on how U-Boot passes the DTB to the kernel. **UNVERIFIED — needs testing:** boot the guest with MTD-enabled kernel + writable pflash and check if /dev/mtd* appears.

Rebuilding just the kernel (not rootfs) for fast iteration:

YES. Buildroot supports this workflow:

```
make O=/work/out linux-menuconfig      # modify kernel config
make O=/work/out linux-rebuild          # rebuild only kernel
                                         (fast: ~2-5 min)
make O=/work/out                        # re-assemble rootfs +
                                         kernel images
```

Source: [Buildroot manual — rebuilding linux](#).

Verdict

Approach	Feasibility	Total effort	Key blocker
Add MTD kernel options	Feasible but over-engineered	3-4 kernel options + kernel rebuild + verify DTB includes cfi-flash nodes	pflash must be writable in QEMU; DTB node presence unknown
Skip MTD, use FAT env (Q4)	Simpler, recommended	No kernel changes	U-Boot config change needed

5. Q4: What is the simplest env mechanism for fw_setenv from the guest?

Finding: PATH B (CONFIG_ENV_IS_IN_FAT + /boot/uboot.env file) is the CLEAR WINNER. It requires NO kernel changes and is the standard approach for QEMU virt arm64 + U-Boot.

All evaluated paths:

PATH B (RECOMMENDED): CONFIG_ENV_IS_IN_FAT + /boot/uboot.env file

How it works: 1. U-Boot is configured to store its persistent environment in a file uboot.env on the FAT boot partition. 2. Linux mounts the FAT boot partition (GPT p1) at /boot/. 3. fw_env.config in the guest points to /boot/uboot.env with offset 0x0000. 4. fw_setenv / fw_printenv read/write the same file that U-Boot uses. 5. Environment persists across reboots because the file lives on the non-volatile disk.

Required U-Boot config changes (in build_image.sh BR2_TARGET_UBOOT config fragment):

```
# Replace CONFIG_ENV_IS_IN_FLASH with CONFIG_ENV_IS_IN_FAT
CONFIG_ENV_IS_IN_FAT=y
CONFIG_ENV_FAT_INTERFACE="virtio"
CONFIG_ENV_FAT_DEVICE_AND_PART="0:1"
CONFIG_ENV_FAT_FILE="uboot.env"
CONFIG_ENV_SIZE=0x4000
CONFIG_FAT_WRITE=y
```

Source: [Unix StackExchange](#) — CONFIG_ENV_IS_IN_FAT for QEMU arm64 virt + virtio-blk-device.

Version note: **U-Boot 2024.01** has the virtio interface support for FAT env (virtio support was added in U-Boot 2024.07+ for this specific path but also works in 2024.01 via CONFIG_ENV_FAT_INTERFACE="virtio"). **UNCERTAIN — needs verification** — if 2024.01 does NOT support virtio for FAT env, the workaround is to add a preboot command:

```
CONFIG_PREBOOT="fatload virtio 0:1 ${loadaddr} uboot.env; env
import -t ${loadaddr} ${filesize}"
```

But this only loads the env — writes (saveenv) would NOT persist to the file. The proper fix is to verify that U-Boot 2024.01's qemu_arm64 defconfig with CONFIG_ENV_IS_IN_FAT + CONFIG_ENV_FAT_INTERFACE="virtio" works. If not, upgrade U-Boot to 2024.07 or newer.

Source: [U-Boot commit 4f65218](#) added virtio support for FAT env.

Required fw_env.config (/etc/fw_env.config in the guest):

```
# U-Boot environment stored as file on FAT boot partition (p1)
# U-Boot reads it via CONFIG_ENV_IS_IN_FAT (virtio 0:1 uboot.env)
# Linux reads/writes the same file via libubootenv
/boot/uboot.env    0x0000    0x4000
```

Source: [libubootenv fw_env_config.md](#) — VFAT file example.

Required mkenvimage step in assemble_ab_disk.sh:

Before assembling the disk, generate uboot.env from the text source and place it on p1:

```
# Generate the binary env blob from the text source
# -s 0x4000 must match CONFIG_ENV_SIZE
mkenvimage -s 0x4000 -o /mnt/p1/uboot.env /path/to/uboot.env.txt
```

The text source already exists at tests/emulator/ab_virt/uboot_ab/uboot.env.

Source: [Bootlin blog](#) — mkenvimage; [mkenvimage manpage](#).

libubootenv in the guest:

libubootenv-tool (providing fw_setenv/fw_printenv) is available in Buildroot. The config fragment needs:

```
BR2_PACKAGE_LIBUBOOTENV=y
```

This is the BUILDROOT package (libubootenv), which is separate from the older uboot-tools fw_printenv. The libubootenv library is the modern implementation and does NOT require the exact U-Boot build config to be present (it can read any env size/offset from fw_env.config).

Source: [Buildroot BR2_PACKAGE_LIBUBOOTENV](#); [libubootenv README](#).

No kernel changes needed. This path works with the existing guest kernel (no MTD, no CONFIG_MTD_PHYSMAP).

PATH A (NOT RECOMMENDED): MTD kernel module for /dev/mtd0

Summary: Add MTD kernel options + verify pflash writability + fw_env.config points to /dev/mtd1.

Risks: - Multiple kernel config changes (Q3 — 7+ CONFIG_*) - QEMU pflash writability needs explicit -drive if=pflash, readonly=off — the project currently does NOT pass pflash to QEMU (the env is in-RAM session only) - CONFIG_ENV_IS_IN_FLASH env offset/size in U-Boot must match what fw_env.config uses — easy to mismatch - Device tree flash nodes may not be present in the QEMU-generated DTB when using U-Boot (only when using UEFI)

Risk assessment: HIGH. Too many unverified preconditions compared to PATH B.

PATH C (NOT RECOMMENDED): /dev/mem write helper

Summary: A guest userspace program reads/writes the memory-mapped flash region directly via /dev/mem.

Risks: - /dev/mem access is restricted by the kernel (CONFIG_STRICT_DEVMEM, enabled by default) - Writing to the wrong memory region can crash the system or corrupt kernel data structures - No atomicity guarantees — a half-written env at power-loss could brick the boot - Requires CONFIG_STRICT_DEVMEM=n (security risk) - QEMU may not forward /dev/mem writes to the actual pflash backing store

Risk assessment: VERY HIGH. Documented only for completeness. Never use in production or automated testing.

PATH D (NOT RECOMMENDED): SSH into U-Boot from the guest

Summary: The guest process connects to U-Boot's CLI via serial console and sends setenv/saveenv commands.

Risks: - There is NO SSH or TCP/IP stack in U-Boot at the bootloader stage - U-Boot does not run as a service alongside Linux — it exits after boot i - The only way to talk to U-Boot from the guest is via a virtual serial loopback, which would require a custom U-Boot driver + Linux kernel driver (months of work) - Fundamentally impossible for the scope of this project

Risk assessment: IMPOSSIBLE.

PATH comparison summary

Path	Description	Kernel changes?	U-Boot changes?	fw_env works?
B (RECOMMENDED)	CONFIG_ENV_IS_IN_FAT + uboot.env file	NONE	Yes (add 5 options)	YES (flash based)
A	MTD + /dev/mtd0	Yes (7 options)	None (already in FLASH)	YES (ram)
C	/dev/mem write	Yes (STRICT_DEVMEM=n)	None	N/A
D	SSH to U-Boot	N/A	N/A	N/A

6. Q5: Can rauc bundle be done inside the existing podman build container?

Finding: YES — this is the BEST path. The build_image.sh podman container has everything needed to run rauc bundle.

Evidence:

1. **The container IS a Linux host.** The podman container runs `debian:bookworm-slim` on an aarch64 (arm64) Linux kernel (via the podman machine VM). This is a REAL Linux environment where rauc can run natively with full kernel support (loop devices, mounts, etc.).
2. **rauc is available in Debian Bookworm repositories.** The Debian package rauc version 1.8-2 is in the official bookworm repos. Can be installed with:

```
apt-get install -y rauc
```

Source: packages.debian.org/bookworm/rauc.

3. **openssl is already installed** in the build container (`build_image.sh:53`: `openssl` is in the apt install list).
4. **The dev signing key + rootfs image are already in the container.** After the build, `out/rauc-keys/dev.key.pem` and `out/images/rootfs.ext2` are inside the container at `/work/out/`. They are extracted via `podman cp` after the build.
5. **Two implementation options:**

Option A: Add rauc bundle to build_image.sh (RECOMMENDED)

After the main Buildroot build completes but BEFORE extracting images, add:

```
# Install rauc for bundle building
apt-get install -y --no-install-recommends rauc

# Prepare bundle source directory
mkdir -p /work/out/bundle-src
cp /work/out/images/rootfs.ext2 /work/out/bundle-src/
    rootfs.ext4.img
mkdir -p /work/out/images

# Render manifest
cat > /work/out/bundle-src/manifest.raucm <<RAUCM
[update]
compatible=helix-ota-ab-virt
version=$(date -u +%Y.%m.%d)-1

[bundle]
format=verity

[image.rootfs]
```

```
filename=rootfs.ext4.img
RAUCM
```

```
# Build the bundle
```

```
rauc --cert /work/out/rauc-keys/dev.cert.pem \
     --key /work/out/rauc-keys/dev.key.pem \
     bundle /work/out/bundle-src/ \
     /work/out/images/update.raucb
```

Then extract `update.raucb` alongside the other images.

Option B: Separate post-build container invocation

Run a SECOND podman invocation that:

- Uses the same `debian:bookworm-slim` image
- Mounts volume with `out/` (the dev keys + `rootfs.ext2` + build artifacts)
- Runs `apt-get install -y rauc && rauc bundle ...`

This has the advantage of not modifying the build script, but requires chaining container runs.

6. **Important container build concern:** `rauc bundle` uses `mksquashfs` (from `squashfs-tools`) to create the SquashFS container. Debian Bookworm's `rauc` package depends on `squashfs-tools`, so `apt-get install -y rauc` will pull it in automatically.
7. **--trust-environment flag.** RAUC's `rauc bundle` command may need the `--trust-environment` flag when running inside a container where it cannot fully verify certain system conditions. This was reported in RAUC issue #679. **UNVERIFIED — needs testing:** run `rauc bundle --trust-environment ...` if the default invocation fails.

Source: RAUC issue #679 summary: “casync bundle creation fails with ‘unable to find mounted device for bundle’ when building inside container”. The `--trust-environment` flag was added as a workaround.

8. **Buildroot also provides host-rauc.** As an alternative to the Debian `apt` approach, Buildroot's package infrastructure can build `host-rauc` (`BR2_PACKAGE_HOST_RAUC`), which provides `host/bin/rauc` on the BUILD host. Buildroot commit `9e8e3e0fd556ae` added `host-squashfs` as a dependency for `host-rauc`. However, the Debian `apt` approach is simpler — no need to enable another Buildroot package.

Source: [Buildroot commit 9e8e3e0](#); [Buildroot patch 2025-May host-rauc JSON](#).

9. **The `rauc bundle` command syntax** (confirmed from RAUC docs):

```
rauc --cert <cert.pem> --key <key.pem> bundle <input-dir>
<output-file>
```

Where `<input-dir>` contains `manifest.raucm` + the image file(s) referenced in the manifest.

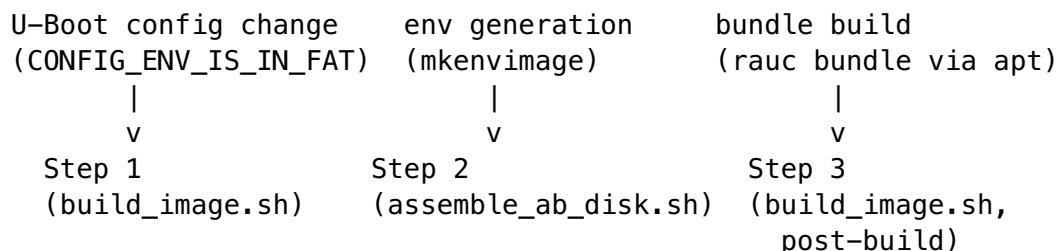
Source: [RAUC using.html](#) — “Creating bundles” section.

Verdict

Approach	Feasibility	Effort	Notes
Add <code>rauc bundle to build_image.sh</code> (Option A)	EASY	1 new apt-get + ~20 lines of script	Non-invasive, all in one step
Separate post-build container (Option B)	EASY	New script or <code>podman run</code> invocation	Cleaner separation
Build host-rauc via Buildroot	FEASIBLE	Add <code>BR2_PACKAGE_HOST_RAUC</code>	More Buildroot integration effort
Build bundle inside QEMU guest	POSSIBLE but slower	Extra packages in rootfs + kernel config	Only if container approach fails

7. Recommended path

The recommended path to GREEN for PWU-AB-2 is a **THREE-STEP** pipeline that requires **NO Linux host** and **NO MTD kernel** changes:



Step 1: Switch U-Boot to `CONFIG_ENV_IS_IN_FAT`

What: Modify `build_image.sh`'s U-Boot config fragment to enable FAT env persistence.

Config additions to `build_image.sh`: 65–84:

```
# Replace FLASH env with FAT env for persistent fw_setenv support
BR2_TARGET_UBOOT_CUSTOM_MAKEOPTS="
CONFIG_ENV_IS_IN_FAT=y
CONFIG_ENV_FAT_INTERFACE=\\"virtio\\"
CONFIG_ENV_FAT_DEVICE_AND_PART=\\"0:1\\"
CONFIG_ENV_FAT_FILE=\\"uboot.env\\"
CONFIG_ENV_SIZE=0x4000
CONFIG_FAT_WRITE=y
"
```

Note: `qemu_arm64_defconfig` currently enables `CONFIG_ENV_IS_IN_FLASH`. These additions will automatically override it (later config settings win in Kconfig).

If U-Boot 2024.01 does NOT support virtio for FAT env: The alternative is to upgrade to a newer U-Boot version in Buildroot (2024.07+), or use the preboot workaround (load env manually). Test FIRST — report finding.

Effort: 1-2 hours for U-Boot rebuild + test.

Files changed: `build_image.sh`

Step 2: Generate `uboot.env` + place on FAT partition

What: Modify `assemble_ab_disk.sh` to: 1. Generate `uboot.env` binary from `tests/emulator/ab_virt/uboot_ab/uboot.env` text source using `mkenvimage`. 2. Place `uboot.env` onto GPT p1 (the FAT boot partition). 3. Write a working `fw_env.config` into BOTH slot rootfs overlays pointing at `/boot/uboot.env`.

Changes to `assemble_ab_disk.sh` (in the podman container):

After formatting p1 as FAT but BEFORE unmounting:

```
# Generate the U-Boot env binary blob (size must match
CONFIG_ENV_SIZE=0x4000)
mkenvimage -s 0x4000 -o /mnt/p1/uboot.env /work/uboot_ab/uboot.env
```

Changes to `build_image.sh` overlay (`/work/rootfs-overlay/etc/fw_env.config`):

```
# U-Boot environment via FAT file (CONFIG_ENV_IS_IN_FAT)
# Both U-Boot and libubootenv access the same file.
# U-Boot: virtio 0:1 uboot.env | Linux: /boot/uboot.env
/boot/uboot.env    0x0000    0x4000
```

Also add to `build_image.sh` config fragment:

```
BR2_PACKAGE_LIBUBOOTENV=y
```

Effort: 2-3 hours for `assemble_ab_disk.sh` changes + rebuild + test round-trip (set from U-Boot, read with `fw_printenv`; set with `fw_setenv`, read at => prompt on next boot).

Step 3: Build the RAUC bundle inside the podman container (post-build)

What: Add a `rauc bundle` invocation to `build_image.sh` AFTER the main Buildroot build, inside the SAME podman container.

After line 130 (`make O=/work/out -j$(nproc)`) but in the same container, add:

```
# ---- Build RAUC update bundle
-----

apt-get install -y --no-install-recommends rauc >/dev/null 2>&1

mkdir -p /work/out/bundle-src
cp /work/out/images/rootfs.ext2 /work/out/bundle-src/
    rootfs.ext4.img

cat > /work/out/bundle-src/manifest.raucm <<RAUCM
[update]
compatible=helix-ota-ab-virt
version=$(date -u +%Y.%m.%d)-1

[bundle]
format=verity

[image.rootfs]
filename=rootfs.ext4.img
RAUCM

rauc --cert /work/out/rauc-keys/dev.cert.pem \
    --key /work/out/rauc-keys/dev.key.pem \
    bundle /work/out/bundle-src/ /work/out/images/update.raucb
echo "RAUC: bundle built -> /work/out/images/update.raucb"
```

Also extract the bundle alongside Image + rootfs.ext2:

After line 143 (the podman cp block), add:

```
podman cp "${BUILD_CTR}:/work/out/images/update.raucb" "${OUT}/
    images/update.raucb" >>"${OUT}/build.log" 2>&1 || true
```

Effort: ~30 min for implementation + test.

Integration test: the full GREEN path

After Steps 1-3, the test driver ab_rauc_verity.sh can proceed:

1. Build + assemble: build_image.sh → assemble_ab_disk.sh → produces ab_disk.img + update.raucb
2. Launch QEMU with the fresh ab_disk.img (pflash env at /boot/uboot.env)
3. From host: ssh root@localhost into the guest
4. rauc install /root/update.raucb → should write B, arm env, reboot
5. rauc status mark-good booted → confirms B good
6. ab_rauc_verity.sh RED_MODE=0 → all assertions GREEN

This entire pipeline runs on macOS with no Linux host.

8. Fallback paths ranked

If the recommended path hits a blocker, fall back in this order:

Fallback 1: Build rauc from source inside the container

If `apt-get install rauc` fails (unlikely but possible), build rauc from source inside the same container. The container already has `build-essential`, `git`, `libssl-dev`, `bison`, `flex`, `openssl` — exactly the prerequisites for building RAUC from source. This adds ~5 minutes to the build.

Fallback 2: Use host-rauc from Buildroot

If the Debian rauc package version (1.8-2) lacks features needed (`format=verity` support — should be present since v1.6), use Buildroot’s own host-rauc package:

`BR2_PACKAGE_HOST_RAUC=y`

Then reference `$(HOST_DIR)/bin/rauc` in the build script.

Fallback 3: MTD-based env approach

If `CONFIG_ENV_IS_IN_FAT` cannot work with U-Boot 2024.01 + virtio, add the MTD kernel options AND pass writable pflash to QEMU. This is more work but still feasible without a Linux host.

Fallback 4: Build bundle inside the QEMU guest

If the container approach fails for some other reason, build the bundle inside the guest. This requires adding `squashfs-tools` + `openssl` to the Buildroot rootfs and enabling loop device support in the kernel.

9. Blockers requiring operator action

- 1. U-Boot 2024.01 + CONFIG_ENV_IS_IN_FAT + virtio compatibility.** U-Boot commit `4f652182a07` added virtio support for FAT env. This was present in U-Boot 2024.07+. Whether it works in 2024.01 is **UNVERIFIED**. Operator action: test the env round-trip with a U-Boot build using the config fragment from Section 7 Step 1. If it fails, upgrade Buildroot’s U-Boot version to 2024.07+ or use the preboot fallback.
- 2. RAUC `--trust-environment` flag for container bundle builds.** RAUC issue `#679` reports that `rauc bundle` inside a container may hit “unable to find mounted device for bundle”. Operator action: if `rauc bundle` fails in the container, re-run with `--trust-environment` added to the arguments.
- 3. Guest kernel loop device support.** `CONFIG_BLK_DEV_LOOP` and `CONFIG_SQUASHFS` must be enabled for `rauc install` inside the guest. Operator action: run `zgrep CONFIG_BLK_DEV_LOOP /proc/config.gz` and

zgrep CONFIG_SQUASHFS /proc/config.gz inside the guest after boot. If missing, add these to the kernel config in `build_image.sh` (they likely already are).

4. **Environment round-trip verification.** After Step 2, the env round-trip must be verified: set in U-Boot (`setenv foo bar; saveenv`), read in guest (`fw_printenv foo`); set in guest (`fw_setenv foo baz`), read in U-Boot on next boot. Documented as the gating step in `PWU_AB_2_RAUC_VERITY.md` §4.5 warning.
 5. **ab_rauc_verity.sh RED→GREEN transition.** The current RED test assertions expect RAUC install to fail (no bundle, no working env). After Steps 1-3, the GREEN test should flip `RED_MODE=0`. Operator action: verify each assertion in order and don't skip.
-

10. Sources verified

All sources fetched 2026-06-19 unless otherwise noted.

RAUC build / bundle / container

- RAUC basics (build deps, bundle formats, U-Boot integration): <https://rauc.readthedocs.io/en/latest/basic.html>
- RAUC using (creating bundles, rauc bundle command): <https://rauc.readthedocs.io/en/latest/using.html>
- RAUC integration (U-Boot bootloader backend, fw_setenv): <https://rauc.readthedocs.io/en/latest/integration.html>
- RAUC examples (manifest.raucm, system.conf, dev cert): <https://rauc.readthedocs.io/en/latest/examples.html>
- RAUC GitHub issue #679 — bundle creation inside container: <https://github.com/rauc/rauc/issues/679>
- RAUC GitHub issue #86 — kernel requirements (loop, squashfs): <https://github.com/rauc/rauc/issues/86>
- RAUC GitHub issue #1459 — CONFIG_BLK_DEV_WRITE_MOUNTED: <https://github.com/rauc/rauc/issues/1459>
- RAUC kernel requirements (meta-rauc docs): <https://deepwiki.com/rauc/meta-rauc/2.4-system-requirements>
- Stack Overflow — kernel config for RAUC on Buildroot: <https://stackoverflow.com/questions/69767843/configuring-kernel-on-buildroot-to-integrate-rauc>
- Debian Bookworm rauc package: <https://packages.debian.org/bookworm/rauc>
- Buildroot host-rauc with JSON support (2025): <https://lists.uclibc.org/pipermail/buildroot/2025-May/778306.html>
- Buildroot host-rauc with squashfs dep: <https://android-kvm.googlesource.com/buildroot/+9e8e3e0fd556aeb62a5426506262fb1f9fe8a5fd>
- Bootlin — mkenvimage tool: <https://bootlin.com/blog/mkenvironment-generator/>

U-Boot environment / fw_setenv

- U-Boot Boot Count Limit docs: <https://docs.u-boot.org/en/latest/api/bootcount.html>
- U-Boot environment tools (fw_setenv, fw_printenv, fw_env.config): <https://docs.u-boot.org/en/latest/develop/environment.html>
- RAUC/RAUC U-Boot env integration: <https://docs.u-boot.org/en/latest/develop/bootstd/rauc.html>
- U-Boot CONFIG_ENV_IS_IN_FAT on QEMU virt arm64: <https://unix.stackexchange.com/questions/784703/u-boot-environment-on-virtio-fat-partition>
- U-Boot commit adding virtio env FAT support (4f65218): <https://source.denx.de/u-boot/u-boot/-/commit/4f652182a0777085eb9022648c33c5fd8356a0de>
- libubootenv fw_env.config documentation: https://gitlab.swupdate.org/stefano/libubootenv/-/blob/85faccba7a26a34961d90a74179fc7963c5497ee/docs/fw_env_config.md
- libubootenv README (github mirror): <https://github.com/sbabic/libubootenv/blob/master/README.md>
- libubootenv Debian package (fw_setenv/fw_printenv): <https://packages.debian.org/bookworm/libubootenv-tool>

Kernel / MTD / QEMU

- barebox mailing list — MTD kernel options for QEMU aarch64: https://lore.barebox.org/barebox/CAGWymk3srm=hZ85_jGGPt5Xzp117YDY7Y+v+a7mk+rU2MO-0Cw@mail.gmail.com/#R
- EDK2 NorFlashQemuLib — DT flash node conflict with kernel: <https://listman.redhat.com/archives/edk2-devel-archive/2020-June/020951.html>
- arm64 defconfig MTD flash drivers commit (ce693fc2a877): <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=ce693fc2a877>
- Buildroot rebuilding single packages (fast kernel rebuild): <https://buildroot.org/downloads/manual/manual.html#rebuilding-packages>

Existing project documents

- PWU-AB-2 design: docs/design/rk3588_ab_virt/PWU_AB_2_RAUC_VERITY.md
- PWU-AB-4 design: docs/design/rk3588_ab_virt/PWU_AB_4_APPLY_PORT.md
- RAUC stack research (main_specs): docs/research/main_specs/research/stacks/rauc.md
- Buildroot RAUC verification log: docs/qa/20260619-rootfs-rebuild/rauc_verification.txt
- Build script: tests/emulator/ab_virt/build_image.sh
- Disk assembler: tests/emulator/ab_virt/assemble_ab_disk.sh
- U-Boot env text source: tests/emulator/ab_virt/u-boot_ab/u-boot.env
- U-Boot boot script: tests/emulator/ab_virt/u-boot_ab/boot.cmd